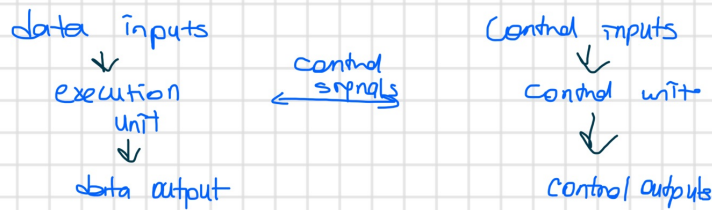


Digital automata

Structure of a typical digital system:



execution unit (datapath)

registers, shifters, ALU, cache memories

Output power elements;
FFs, DACs

Control unit

control data movements by switching multiplexers

execute some program high or lower

Assembly language is used in this unit

One written

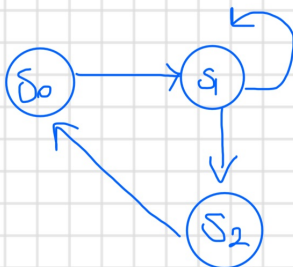
Finite State machine

FSM:

any circuit with memory is a Finite State Machine

Moore

define states
transitions between states
optimization/minimization } design



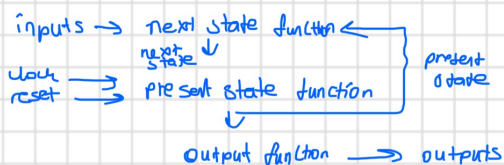
private customer doesn't know what he wants. Want from engineer modify many time specific project. If you implement some basic logical condition (if-else-while) chance of that it will be hard to changing it. You will obtain many problem. functionality. Additional state is enough for digital automata. Every thing is exactly same. We can design new functionality which can not functionality. It is one of biggest reason to use digital automata.

Some initial values are not optimum. That's why we do optimization

Triangular table method!
Minimization same as before.

Moore FSM:

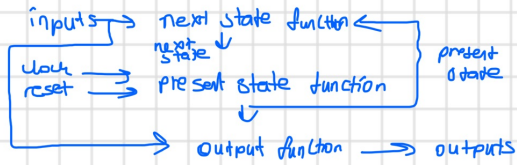
Output is a function of present state only.



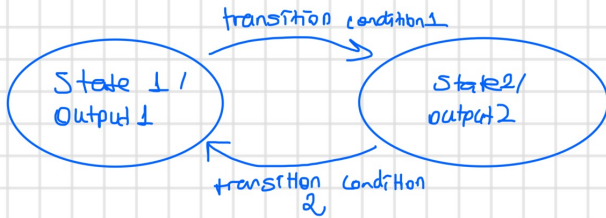
traffic lamps vending machine → power element give drive instance

Mealy FSM:

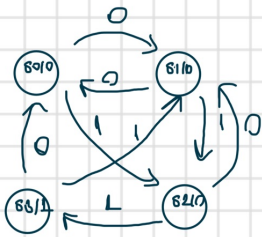
Output is a function present state and inputs



click the button for factory stop whole machine



If we stable on same state the output will be same every time



1st Step:

entity fsm is

Port

x in bit;

clk, reset in bit;

y out bit;

end fsm;

2nd Step:

enumeration states

s0, s1, s2, s3

architecture moore of fsm is

type state is (s0, s1, s2, s3);

(green, yellow, red, ...)

signal current_state, next_state: state;

process (clk, reset)

begin

if reset = '1' then

current_state <= s0;

elsif clk'event and clk = '1' then

current_state <= next_state;

end if;

end process;

process (x, current_state)

begin

case current_state is

y <= '0';

if x = '0' then

next_state <= s1;

else

next_state <= s2;

end if;

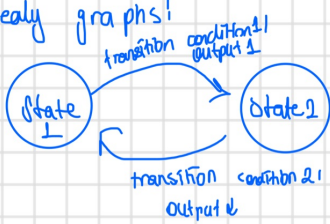
...

type instead of STD_LOGIC

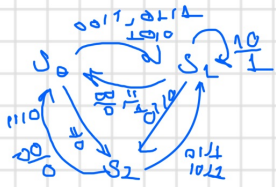
you can do in this way very simple way

it is more hard to understand without new state definition

mealy graphs:



mealy do some examples 2. Input, 1 output.



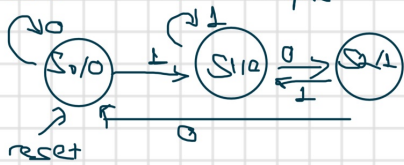
Moore vs Mealy fsm

→ (functionally) can be equivalent

Mealy fsm has richer description and usually requires smaller number of states

separate input from output ⇒ Moore
the reaction is not immediately

Moore fsm example



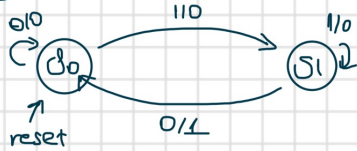
discover
sequence "10"

S₀: no elements
of the
sequence
observed

S₁: observed
"1"

S₂: "10"
observed

Mealy FSM



Sequence: "10"

It has more
simple
graph than
the Moore

States: S₀: no elements of
the sequence
observed

S₁: "1"
Observed